

AMDP 練習 7：CDS Table Function——AMDP 的另一個身分

Lecture

前面六題的 AMDP Method 都是「ABAP 端主動呼叫」——`CALL METHOD` / 函數式呼叫，呼叫端明確知道自己在呼叫一個 AMDP。這題示範 AMDP 的另一種身分：包裝成 **CDS Table Function**，讓呼叫端只看到一個 **CDS Entity**，可以直接用 **Open SQL `SELECT ... FROM`** 查詢，完全不知道背後其實是一段 **SQLScript**。

CDS View vs CDS Table Function 的關鍵差異：一般 CDS View (`DEFINE VIEW`) 只能用宣告式的方式描述查詢 (`SELECT/JOIN/UNION` 等)，複雜到某個程度就描述不出來；**CDS Table Function (`DEFINE TABLE FUNCTION`)** 可以塞任意 **SQLScript 邏輯** (包含 am03 教過的變數/流程控制)，但對外暴露的介面看起來完全像一個普通的 CDS Entity——`SELECT ... FROM ztf_xxx` 就能查，不需要知道背後是 AMDP。這題把 am04 已經驗證過的「航線載客率」邏輯，原封不動包成一個 Table Function，示範同一段邏輯兩種暴露方式的差異。

學習目標

- 會用 `DEFINE TABLE FUNCTION ... RETURNS { ... } IMPLEMENTED BY METHOD ...` 宣告一個 CDS Table Function，並理解它跟 CDS View 的本質差異 (View 只能宣告式查詢，Table Function 可以塞任意 SQLScript)
- 知道實作 **Table Function 的 AMDP 方法要用 `BY DATABASE FUNCTION`**，不是 `BY DATABASE PROCEDURE`——這是這題最重要、也是實測踩到的關鍵語法差異
- 理解 **CDS Table Function 如果底層資料是 Client 相關的，`returns` 結構必須明確宣告一個 `abap.clnt` 型別欄位 (放在第一個位置)**，這樣透過 Open SQL 查詢這個 Table Function 時，ABAP 才會像查詢一般 Client 相關表格一樣自動做 Client 過濾——這是 am01 教訓「AMDP 不會自動處理 Client」在 Table Function 情境下的**例外**：包成 Table Function 之後，透過 Open SQL 查詢反而會自動過濾，只要 `returns` 結構有正確宣告 CLNT 欄位

事前準備

ADT 端物件已由課程準備好 (`$TMP`)：

- `ZTF_AM07_ROUTE_STATS`——CDS Table Function 定義 (DDL Source)
- `ZCL_AM07_ROUTE_STATS`——實作這個 Table Function 的 AMDP 類別，邏輯完全沿用 am04 的航線載客率計算
- `ZR_AM07_DEMO`——demo 程式，用純 **Open SQL** (不是呼叫 AMDP Method) 查詢這個 Table Function

題目需求 (對照已建好的答案物件，含實測踩坑過程)

1. `ZTF_AM07_ROUTE_STATS` (CDS DDL Source)：

```
`abap @EndUserText.label: 'AM07: route load stats table function' define table function
ZTF_AM07_ROUTE_STATS returns { mandt : abap.clnt; carrid : s_carr_id; carrname :
s_carrname; connid : s_conn_id; flight_cnt : abap.int4; seats_occ : abap.int4; seats_max
: abap.int4; load_pct : abap.dec(5,1); } implemented by method
zcl_am07_route_stats=>get_data; ``
```

1. ZCL_AM07_ROUTE_STATS (實作 Table Function 的 AMDP 類別) :

```
``abap CLASS zcl_am07_route_stats DEFINITION PUBLIC FINAL CREATE PUBLIC. PUBLIC SECTION. INTERFACES  
if_amdp_marker_hdb.
```

```
CLASS-METHODS get_data FOR TABLE FUNCTION ztf_am07_route_stats. ENDCLASS.
```

```
CLASS zcl_am07_route_stats IMPLEMENTATION.
```

```
METHOD get_data BY DATABASE FUNCTION FOR HDB LANGUAGE SQLSCRIPT OPTIONS READ-ONLY USING  
scarr sflight.
```

```
RETURN WITH route_totals AS ( SELECT f.mandt, f.carrid, f.connid, COUNT(*) AS flight_cnt, SUM(f.seatsocc) AS  
seats_occ, SUM(f.seatsmax) AS seats_max FROM sflight AS f GROUP BY f.mandt, f.carrid, f.connid ) SELECT  
rt.mandt, rt.carrid, c.carrname, rt.connid, rt.flight_cnt, rt.seats_occ, rt.seats_max, CAST(rt.seats_occ * 100.0 /  
rt.seats_max AS DECIMAL(5,1)) AS load_pct FROM route_totals AS rt JOIN scarr AS c ON c.mandt = rt.mandt  
AND c.carrid = rt.carrid;
```

```
ENDMETHOD.
```

```
ENDCLASS. ``
```

1. 實測踩坑一：returns 結構第一版沒有 CLNT 欄位，啟用失敗：

第一版 returns 只放業務欄位 (carrid/carrname/...，沒有 mandt)，啟用時系統回報：

```
> ZTF_AM07_ROUTE_STATS is marked as client-specific; type field CARRID at pos. 1 is CHAR  
(not CLNT)
```

系統自動偵測到這個 Table Function 底層用到 Client 相關的表 (SCARR/SFLIGHT)，判定它「應該是 Client 相關的」，因此要求 returns 結構的第一個欄位必須是 abap.clnt 型別，用來當作 Client 判別欄位。修正方式：在 returns 最前面加上 mandt : abap.clnt；AMDP 方法的 RETURN SELECT 也要對應把 rt.mandt 放在欄位清單第一位。

1. 實測踩坑二：FOR TABLE FUNCTION 寫在 IMPLEMENTATION 的方法本體宣告，啟用失敗：

第一版把 IMPLEMENTATION 裡的方法本體寫成 METHOD get_data BY DATABASE PROCEDURE FOR TABLE FUNCTION ... (複製 CLASS DEFINITION 那行的 FOR TABLE FUNCTION 語法)，啟用回報："HDB LANGUAGE SQLSCRIPT" expected after "FOR"。——FOR TABLE FUNCTION <cds名稱> 只出現在 CLASS DEFINITION 的方法宣告 (CLASS-METHODS get_data FOR TABLE FUNCTION ztf_...)，IMPLEMENTATION 裡的方法本體宣告維持 FOR HDB LANGUAGE SQLSCRIPT，兩處語法不一樣，容易搞混。

1. 實測踩坑三 (也是最重要的一個)：BY DATABASE PROCEDURE 用在 Table Function 實作方法上啟用失敗：

修完踩坑二之後，啟用回報一個更精確的錯誤：The method "GET_DATA" implements the CDS table function "ZTF_AM07_ROUTE_STATS", but "GET_DATA" is not a database function。——實作 CDS Table Function 的 AMDP 方法，關鍵字要用 BY DATABASE FUNCTION，不是前六題一路用的 BY DATABASE PROCEDURE。這是 AMDP 框架區分「一般被 ABAP 呼叫的資料庫程序 (Procedure)」跟「被 CDS Table Function 呼叫的資料庫函數 (Function)」的方式——兩者的 SQLScript 語法幾乎一樣 (這題一樣用 WITH

CTE、JOIN)，但 ABAP 端的宣告關鍵字不同，而且 Table Function 版本本體最後用 **RETURN SELECT ...;**（單一結果集，沒有 **EXPORTING** 參數的概念），不是前面幾題 **et_xxx = SELECT ...;** 的寫法。

1. **ZR_AM07_DEMO**（純 Open SQL 查詢，完全不呼叫 AMDP Method）：

```
``abap REPORT zr_am07_demo.
```

```
SELECT carrid, carrname, connid, flight_cnt, seats_occ, seats_max, load_pct FROM ztf_am07_route_stats ORDER BY carrid, connid INTO TABLE @DATA(lt_routes).
```

```
LOOP AT lt_routes ASSIGNING FIELD-SYMBOL(<ls_route>). WRITE: / <ls_route>-carrid, <ls_route>-carrname, <ls_route>-connid, <ls_route>-flight_cnt, <ls_route>-seats_occ, <ls_route>-seats_max, <ls_route>-load_pct. ENDLOOP.
```

```
WRITE: / 'Total rows:', lines( lt_routes ). ``
```

注意這支程式完全沒有出現 **ZCL_AM07_ROUTE_STATS** 這個類別名稱——呼叫端只認得

ZTF_AM07_ROUTE_STATS 這個 CDS Entity，用一般 **SELECT ... FROM** 語法查詢，這正是把 AMDP 包成 Table Function 的意義：對外暴露成一個看起來完全普通的資料來源。

預期輸出（實測畫面）

AA	American Airlines	0017	12	2,880	4,620	62.3
AA	American Airlines	0064	13	2,832	4,290	66.0
...						
LH	Lufthansa	0400	16	3,220	5,350	60.1
...						
UA	United Airlines	3517	15	3,932	5,775	68.0
Total rows:		26				

跟 **am04** 的實測結果完全一致（26 筆航線、每筆數字都相同）——這證明兩件事：(1) 包成 Table Function 沒有改變底層邏輯的正確性；(2) **Total rows: 26** 代表透過 **Open SQL** 查詢時，**Client** 過濾正確生效，只看到目前 Client（130）的資料，沒有像 **am01** 一開始那樣因為多 Client 而重複。

團隊實務備註

- 這是 **am01** 核心教訓的一個重要例外：am01 說「AMDP 不會像 Open SQL 自動處理 Client」，這句話對「直接呼叫 AMDP Method」成立，但對「透過 **CDS Table Function + Open SQL 查詢**」不成立——只要 **returns** 結構正確宣告了 CLNT 欄位，ABAP 的 Open SQL 編譯器會把這個 Table Function 當作一般 Client 相關的資料來源，自動加上 Client 過濾。這不代表 SQLScript 本體「學會了」自動過濾（**ZCL_AM07_ROUTE_STATS** 內部的 SQLScript 一樣沒有寫死過濾邏輯，是 CDS 框架層在 Table Function 外面多包了一層），而是過濾發生的位置從「你自己在 **SQLScript** 裡手動寫」變成「**CDS 框架層**在你宣告了 **CLNT** 欄位之後自動幫你做」
- 三個踩坑（**CLNT** 欄位、**FOR TABLE FUNCTION** 只在 **DEFINITION**、**BY DATABASE FUNCTION** 不是 **BY DATABASE PROCEDURE**）全部是編譯期就會擋下來的，而且系統給的錯誤訊息都直接點出問題所在（例如第三個坑直接說「不是一個 database function」），這是這門課到目前為止錯誤訊息最明確的一次，不需要太多推理就能找到修正方向

- 這題兩個物件 (**DDL Source + AMDP 類別**) 互相引用，只能一起批次啟用：ZTF_AM07_ROUTE_STATS 的 **implemented by method** 指向還沒啟用的 ZCL_AM07_ROUTE_STATS，ZCL_AM07_ROUTE_STATS 的 **FOR TABLE FUNCTION** 指向還沒啟用的 ZTF_AM07_ROUTE_STATS——先啟用任一個都會因為對方不存在而失敗，這種「兩個物件互相依賴」的情境要在同一次 **activation** 請求裡把兩個物件都放進去，系統會自動處理相依順序 (呼應 [.claude/rules/sap-adt-mcp.md](#) 第 5 節提過的「多物件可以在同一個 activation 請求批次啟用」)

思考題

1. 如果把 **returns** 結構的 **mandt : abap.clnt;** 拿掉、改成手動在 AMDP 方法裡用 **iv_mandt** 參數 (像 am01~am06 一樣)，這樣的 Table Function 還能不能被純 Open SQL **SELECT ... FROM ztf_xxx** 查詢？ (提示：**with parameters** 語法可以讓 Table Function 接受輸入參數，但呼叫端查詢時就要寫成 **SELECT ... FROM ztf_xxx(p_xxx = ...)**，跟這題完全不帶參數、靠 CLNT 欄位自動過濾的寫法是兩種不同的設計)
2. 這題的 AMDP 方法本體幾乎跟 am04 一模一樣 (同一段 CTE + JOIN)，只差在方法宣告用 **BY DATABASE FUNCTION** 而不是 **BY DATABASE PROCEDURE**、多了 **mandt** 欄位——這是不是代表以後每個 AMDP Method 都應該優先包成 Table Function，反正多包一層也沒什麼壞處？有沒有什麼情境是「一定要用直接呼叫的 AMDP Method，不適合包成 Table Function」的？ (提示：回想 am02 教過的「同時 EXPORTING 兩個以上 Internal Table」——Table Function 的 **returns** 只能定義一種結果集形狀，這種需求還做得到嗎？)
3. ZR_AM07_DEMO 的 Open SQL 可以加 **WHERE carrid = 'LH'** 這種條件嗎？這個 **WHERE** 條件會被下推到 SQLScript 裡執行 (在資料庫端就篩選好)，還是查出全部 26 筆之後才在 ABAP 端篩選？這對效能有什麼影響？

答案

見 **ztf_am07_route_stats.ddls.asddls**、**zcl_am07_route_stats.clas.abap**、**zr_am07_demo.prog.abap** (SAP 端物件 ZTF_AM07_ROUTE_STATS / ZCL_AM07_ROUTE_STATS / ZR_AM07_DEMO，套件 \$TMP)。